

Phase 5: Project Development Phase

Project Name: **FitSense (FitTrack AI)** — Modular Implementation, Code Logic & AI Integration

5.1 Development Methodology & Engineering Strategy

The development phase of FitSense adheres to a modular, layered engineering strategy following the **Model-View-Controller (MVC)** paradigm. The system architecture separates application entry points, middleware interceptors, routing definitions, domain controllers, and persistent Mongoose schemas. Core development efforts prioritized secure cryptographic handling, reliable MongoDB connections, clean input validation, and asynchronous integration with Google Gemini AI.

5.2 Environment Initialization & Dependency Configuration

Project Workspace & Package Setup

Executed from the designated server directory:

```
# Directory initialization mkdir AI-FitTrack-API && cd AI-FitTrack-API mkdir server && cd server # Initialize Node manifest & install production packages npm init -y npm install express mongoose dotenv bcryptjs jsonwebtoken cors # Install development daemon npm install --save-dev nodemon
```

Environment Configuration (.env)

Key secrets and operational configurations isolated from source code:

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/aifittrack
JWT_SECRET=your_secret_key
GEMINI_API_KEY=your_google_gemini_api_key
```

Isolation protects credentials from exposure while ensuring dynamic configuration across testing and deployment.

5.3 Database & Data Model Implementation

Database Connector: config/db.js

Asynchronous MongoDB connection utilizing Mongoose ODM:

```
const mongoose = require('mongoose');
const connectDB = async () => { try { const conn = await mongoose.connect(process.env.MONGO_URI); console.log(`MongoDB Connected: ${conn.connection.host}`); } catch (error) { console.error(`Error connecting to MongoDB: ${error.message}`); process.exit(1); } };
module.exports = connectDB;
```

Cryptographic Password Hook: models/User.js

Mongoose pre-save hook enforcing bcrypt hashing with 10 salt rounds:

```
userSchema.pre('save', async function (next) { if (!this.isModified('password')) { return next(); } const salt = await bcrypt.genSalt(10); this.password = await bcrypt.hash(this.password, salt); next(); });
userSchema.methods.matchPassword = async function (enteredPassword) { return await bcrypt.compare(enteredPassword, this.password); };
```

5.4 Workout Schema Constraints & Validation Implementation

Implemented in `models/Workout.js`, enforcing validation boundaries before persistence:

- **Category Bounds:** Restricted to enum: ['Cardio', 'Strength Training', 'Yoga', 'Running', 'Cycling', 'Walking'] with custom rejection messaging.
- **Numeric Bounds:** Enforces duration with min: [1, 'Duration must be at least 1 minute'] and caloriesBurned with min: [0, 'Calories burned cannot be negative'].
- **Referential Integrity:** Binds the workout directly to the creator via user: { type: mongoose.Schema.Types.ObjectId,

5.5 Backend Module-Wise Implementation Breakdown

Component	Source File	Implementation Details & Logic
Server Entry	server.js / app.js	Initializes Express, activates cors(), applies JSON body parsers, registers base routes (/api/auth, /api/workouts, /api/ai), and attaches global error handling.
Auth Middleware	middleware/auth.js	Intercepts private requests, extracts Bearer token from Authorization header, verifies signature using jwt.verify(), and assigns user context to req.user.
Auth Controller	controllers/authController.js	Handles user registration, verifies unique email addresses, coordinates password authentication via matchPassword(), and signs stateless JWT tokens.
Workout Controller	controllers/workoutController.js	Executes CRUD methods (addWorkout, getWorkouts, getWorkoutById, updateWorkout, deleteWorkout); manages multi-field regex search filtering.
AI Controller	controllers/aiController.js	Extracts payload parameters (demographics and workout metrics), formats prompt instructions, invokes Gemini service routines, and sends JSON responses.
AI Service Layer	services/geminiService.js	Encapsulates the Google Gemini API client; formats generative prompts, executes external HTTPS requests, and extracts text responses.

5.6 Google Gemini AI Integration Logic

Workout Recommendation Integration Constructs generative fitness plans using demographic criteria: Input: age, fitnessGoal, experience Prompt Flow: Sends contextual system prompt to Gemini requesting split breakdown, volume, exercises, and safety warnings. Output: Standardized text payload containing structured weekly training recommendations.	Fitness Insights Integration Generates analytical performance summaries from recorded logs: Input: totalWorkouts, averageDuration, totalCaloriesBurned Prompt Flow: Directs Gemini to evaluate cumulative work, assess training consistency, and suggest progressive overload steps. Output: Concise motivational assessment and performance feedback.
--	---

5.7 Integration Architecture & Layer Scope

Frontend Implementation <i>Not specified in the project document.</i> The codebase implements a headless RESTful API backend. Architectural interfaces are decoupled to allow frontend consumers (e.g., React, native mobile apps) and API testing suites (Postman, Thunder Client) to interact via HTTP JSON.	API / Integration Scope Pure JSON endpoints conforming to REST specifications. Inter-module communication relies on asynchronous promises (async/await), standard HTTP status codes (200, 201, 400, 401, 500), and centralized error handling.
--	--

5.8 Development Challenges and Technical Solutions Documented Solutions: Centralized error middleware was introduced to catch validation failures, CastErrors, and token expirations uniformly. Asynchronous Mongoose schemas ensure that passwords are never persisted in plaintext, while modular service classes isolate Google Gemini API calls from core routing logic. Additional Challenges: <i>Not specified in the project document</i> beyond the documented architectural patterns and defensive validations.
